

PRAKTIKUM 1

CLASS, OBJECT, CONSTRUCTOR dan METHOD

Interaksi Objek & Manipulasi State (Mini Bank)

1.1 Tujuan

- Mendefinisikan class sebagai model dari entitas dunia nyata
- Mendefinisikan object dan memahami alokasi memori independent
- Menggunakan constructor untuk inisialisasi awal state
- Manipulasi state (atribut) melalui behavior (method) menggunakan interaksi CLI

1.2 Alat

- Computer / laptop yang telah terinstall JDK dan Eclipse

1.3 Teori

Dalam Pemrograman Berorientasi Objek (PBO), perangkat lunak dimodelkan berdasarkan entitas di dunia nyata. Dua pilar paling mendasar dalam pemodelan ini adalah Class dan Object.

- **Class sebagai Blueprint**, Menurut Deitel & Deitel (2017) dalam bukunya Java: How to Program, sebuah class adalah cetak biru (blueprint) dari sebuah objek. Class mendefinisikan struktur data (attributes/state) dan sekumpulan operasi (methods/behavior) yang dapat dilakukan pada data tersebut. Class sendiri tidak memakan ruang memori untuk menyimpan nilai data, melainkan hanya berfungsi sebagai tipe data referensi.
- **Object dan State**, Objek adalah instansiasi konkret dari sebuah class yang dialokasikan di dalam memori komputer. Setiap objek memiliki state/kondisi yang independen. Sierra & Bates (2005) dalam Head First Java mengilustrasikan bahwa dua objek dari class yang sama misalnya Rekening dapat memiliki nilai state seperti saldo yang sama sekali berbeda, dan perubahan pada satu objek tidak akan memengaruhi objek lainnya.
- **Constructor** adalah blok kode khusus yang dieksekusi secara otomatis pada saat objek dibuat menggunakan keyword **new**. Fungsinya adalah untuk menginisialisasi state awal dari objek tersebut (Oracle, 2023).

Dalam konteks perbankan, Class adalah konsep "**Rekening**", Object adalah "**Rekening milik Mawar**" atau "**Rekening milik Melati**", State adalah "**Saldo saat ini**", dan Behavior adalah aktivitas seperti "**Setor Tunai**" yang akan memanipulasi state saldo tersebut.

1.4 Langkah-langkah

Pada praktikum ini, kita akan membuat *Core Engine* perbankan tanpa antarmuka grafis (GUI), melainkan menggunakan *Command Line Interface* (CLI) agar fokus pada logika memori.

Membuat Class Rekening

Buatlah sebuah file bernama **Rekening.java**. Class ini bertindak sebagai model utama.

1. Buat class Rekening

```
public class Rekening
{
}
```

2. Tambahkan State/Atribut

```
String nomorRekening;
String namaPemilik;
double saldo;
```

3. Buat constructor

```
public Rekening(String nomor, String nama, double saldoAwal) {
    nomorRekening = nomor;
    namaPemilik = nama;
    saldo = saldoAwal;
    System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
}
```

4. Buat method / behavior

```
public void setorTunai(double nominal) {
    if (nominal > 0) {
        saldo += nominal;
        System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
    } else {
        System.out.println("Gagal: Nominal setor harus lebih dari 0!");
    }
}

public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir : Rp" + saldo);
    System.out.println("-----");
}
```

Kode program lengkap untuk class **Rekening**

```

public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    public Rekening(String nomor, String nama, double saldoAwal) {
        nomorRekening = nomor;
        namaPemilik = nama;
        saldo = saldoAwal;
        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
        }
    }

    public void cekInformasi() {
        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir : Rp" + saldo);
        System.out.println("-----");
    }
}

```

Membuat Engine CLI pada Main.java

Buat file **Main.java**. pada class ini kita akan membuat bagaimana interaksi *user* memanggil sebuah objek dengan menggunakan fungsi **Scanner** dan **while-loop**.

NURFIAH, S.ST, M.KOM

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Rekening akunAktif = null; // Objek belum diinisialisasi (null)
        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while (isRunning) {
            System.out.println("\nMenu Utama:");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");

            int pilihan = input.nextInt();
            input.nextLine(); // Membersihkan buffer enter

            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan No Rekening: ");
                    String no = input.nextLine();
                    System.out.print("Masukkan Nama Pemilik: ");
                    String nama = input.nextLine();
                    System.out.print("Masukkan Saldo Awal: ");
                    double saldo = input.nextDouble();

                    // Instansiasi Object / Menjalankan Constructor
                    akunAktif = new Rekening(no, nama, saldo);
                    break;

                case 2:
                    if (akunAktif == null) {
                        System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
                    } else {
                        System.out.print("Masukkan nominal setor: ");
                        double setor = input.nextDouble();
                        akunAktif.setorTunai(setor); // Memanggil Behavior / method
                    }
                    break;

                case 3:
                    System.out.println("Fitur ini akan kerjakan sebagai Tugas Mandiri.");
                    break;

                case 4:
                    if (akunAktif == null) {
                        System.out.println("Error: Anda belum membuka rekening!");
                    } else {
                        akunAktif.cekInformasi();
                    }
                    break;

                case 0:
                    isRunning = false;
                    System.out.println("Sistem ditutup. Terima kasih!");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
            }
        }
        input.close();
    }
}

```

Jalankan program menggunakan CLI dan lakukan ujicoba seluruh fitur yang ada

1.5 Latihan /Tugas

Untuk menguji pemahaman dalam melakukan manipulasi *state* (saldo) dan pengelolaan banyak objek secara dinamis, selesaikan tugas berikut:

1. Implementasi Fitur Tarik Tunai

- Tambahkan method **tarikTunai(double nominal)** ke dalam **class Rekening**.
- Aturan : Saldo tidak boleh menjadi **minus** dan minimal nominal Tarik tunai adalah **10.000**. Jika pengguna mencoba menarik nominal yang lebih besar dari saldo saat ini atau nominal kecil dari 10.000, program tidak boleh crash/error, melainkan menampilkan peringatan ke konsol: "Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp[nominal]" atau "Transaksi Gagal : Minimal nomonal penarikan 10.000".
- Terapkan method ini dengan menu nomor 3 pada **class Main**.

2. Tantangan Multi-Akun

Saat ini, aplikasi **Main** hanya dapat menyimpan 1 akun (**Rekening akunAktif = null;**). Jika pengguna memilih menu "**Buka Rekening Baru**" untuk kedua kalinya, akun lama akan tertimpa.

- Ubah arsitektur penyimpanan di **Main.java** menggunakan struktur data **Collection**. (**Gunakan ArrayList<Rekening>**).
- Ubah logika menu "**Buka Rekening**": Setiap kali pengguna membuka rekening baru, objek tersebut ditambahkan ke dalam **ArrayList**.
- (Bonus): Tambahkan menu baru untuk "**Ganti Akun**", di mana sistem akan meminta nomor rekening, kemudian mencari objek yang cocok di dalam **ArrayList** untuk dijadikan **akunAktif**